P3296R1 `let_async_scope`

Date: 20th June 2024

Author: Anthony Williams anthony@justsoftwaresolutions.co.uk

Audience: SG1, LEWG

Background and motivation

This is intended to address concerns raised in LEWG about ensuring that a `counting_scope` (see P3149R3) is joined: the scope provided by `let_async_scope` is always joined, irrespective of how the nested work completes, and whether or not the provided function throws an exception after spawning work.

Code with explicit `counting_scope`:

```
some_data_type scoped_data = make_scoped_data();
counting_scope scope;

spawn(scope,(on(exec, [&] {
    spawn(scope,(on(exec, [&] {
        if (need_more_work(scoped_data)) {
            spawn(scope,(on(exec, [&] { do_more_work(scoped_data); })));
            spawn(scope,(on(exec, [&] { do_more_other_work(scoped_data); })));
        }
    })));
    spawn(scope,(on(exec, [&] { do_something_else_with(scoped_data); })));
})));

maybe_throw();

this_thread::sync_wait(scope.join());
```

Here, if `maybe_throw` throws an exception, then the scope is not joined, and the nested tasks can continue executing asynchronously, potentially accessing both the `scope` and `scoped_data` objects out of lifetime.

Using `let_async_scope` addresses this by encapsulating the scope object and the result of the previous sender. The returned sender does not complete until all tasks nested on the scope complete, even if the function passed to `let_async_scope` exits via an exception:

```
auto scope_sender = just(make_scoped_data()) | let_async_scope([](auto scope_token,
                                                                 auto& scoped_data) {
    spawn(scope_token,(on(exec, [scope_token, &scoped_data] {
        spawn(scope_token,(on(exec, [scope_token, &scoped_data] {
            if (need_more_work(scoped_data)) {
                spawn(scope_token,(on(exec, [&scoped_data] { do_more_work(scoped_data); })));
                spawn(scope_token,(on(exec, [&scoped_data] { do_more_other_work(scoped_data); })));
            }
        })));
        spawn(scope_token,(on(exec, [&scoped_data] { do_something_else_with(scoped_data); })));
    })));
    maybe_throw();
});

this_thread::sync_wait(scope_sender);
```

Here, even if `maybe_throw` throws an exception, then `scope_sender` doesn't complete until all the nested tasks have completed. This prevents out-of-lifetime access to the `scoped_data` or the scope itself, unless references to the data or `scope_token` are stored outside the sender tree.

Stop requests are propagated to all senders nested in the async scope, but does not prevent those senders adding additional work to the scope. This allows senders to respond to stop requests by scheduling additional work to perform the necessary cleanup for cancellation.

Proposal

`let_async_scope` provides a means of creating an async scope (see P3149R3), which is associated with a set of tasks, and ensuring that they are all complete before the async scope sender completes.The previous sender's result is passed to a user-specified invocable, along with an async scope token, which returns a new sender that is connected and started.

The sender returned by `let_async_scope` completes with the result of the completion of the sender returned from the supplied invocable. It does not complete until all tasks nested on the `scope_token` passed to the invocable have completed. Additional tasks may be nested on copies of the `scope_token`, even if the initial sender returned from the invocable has completed. The returned `scope_sender` will not complete while there are any nested tasks that have not completed.

Stop requests are propagated to all senders nested in the async scope.

Wording

`execution::let_async_scope`

1. `let_async_scope` transforms a sender's value completions into a new child asynchronous operation associated with an async scope, by passing the sender's result datums to a user-specified callable, which returns a new sender that is connected and started.

2. For a subexpression `sndr`, let `let-async-scope-env(sndr)` be expression-equivalent to the first well-formed expression below:

   - `SCHED-ENV(get_completion_scheduler<decayed-typeof<set_value>>(get_env(sndr)))`

   - `MAKE-ENV(get_domain, get_domain(get_env(sndr)))`

   - `empty_env{}`

3. The expression `let_async_scope(sndr, f)` is expression-equivalent to:

```
transform_sender(
  get-domain-early(sndr),
  make-sender(let_async_scope, f, sndr));
```

4. The exposition-only class template `impls-for` ([exec.snd.general]) is specialized for `let_async_scope` as follows:

```
namespace std::execution {
  template<class State, class Rcvr, class... Args>
  void let-async-scope-bind(State& state, Rcvr& rcvr, Args&&... args); // exposition only

  template<>
  struct impls-for<decayed-typeof<let_async_scope>> : default-impls {
    static constexpr auto get-state = see below;
    static constexpr auto complete = see below;
  };
}
```

   1. Let `receiver2` denote the following exposition-only class template:

```
namespace std::execution {
  template<class Rcvr, class Env>
  struct receiver2 : Rcvr {
    explicit receiver2(Rcvr rcvr, Env env)
      : Rcvr(std::move(rcvr)), env(std::move(env)) {}

    auto get_env() const noexcept {
      const Rcvr& rcvr = *this;
      return JOIN-ENV(env, FWD-ENV(execution::get_env(rcvr)));
    }

    Env env; // exposition only
  };
}
```

2. `impls-for<decayed-typeof<let_async_scope>>::get-state` is is initialized with a callable object equivalent to the following:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) requires see below {
  auto&& [tag, data, child] = std::forward<Sndr>(sndr);
  return [&]<class Fn, class Env>(Fn fn, Env env) {
    using args-variant-type = see below;
    using ops2-variant-type = see below;
    using scope-type = see below;

    struct state-type {
      Fn fn;
      Env env;
      scope-type scope;
      args-variant-type args;
      ops2-variant-type ops2;
    };
    return state-type{std::move(fn), std::move(env), {}, {}};
  }(std::forward_like<Sndr>(data), let-async-scope-env(child));
}
```

   1. `scope-type` is a type that satisfies the `asynchronous-scope` concept.

      1. Instances of `scope-type` maintains a count of the number of active senders passed to `nest` on that scope object or an `async-scope-token` obtained from that scope object.
      2. The count of active senders is decremented by one when a sender passed to `nest` completes.
      3. The sender returned from calling `join()` on an instance of the scope type will complete when the number of senders reaches zero.
      4. Once the count of active senders has been decremented to zero, it is undefined behaviour to attempt to nest a sender into the scope. [[Note: this requires storing a scope token passed to a nested sender in storage that outlives that sender]]

   2. `scope-token-type` is the type of the `async-scope-token` returned from `scope-type::get_token`.

   3. Let `Sigs` be a pack of the arguments to the `completion_signatures` specialization named by `completion_signatures_of_t<child-type<Sndr>, env_of_t<Rcvr>>`. Let `LetSigs` be a pack of those types in `Sigs` with a return type of `decayed-typeof<set_value>`. Let `as-tuple` be an alias template such that `as-tuple<Tag(Args...)>` denotes the type `decayed-tuple<Args...>`. Then `args-variant-type` denotes the type `variant<monostate, as-tuple<LetSigs>...>`.

   4. Let `as-sndr2` be an alias template such that `as-sndr2<Tag(Args...)>` denotes the type `call-result-t<Fn, scope-token-type, decay_t<Args>&...>`. Then `ops2-variant-type` denotes the type `variant<monostate, connect_result_t<as-sndr2<LetSigs>, receiver2<Rcvr, Env>>...>`.

   5. The *requires-clause* constraining the above lambda is satisfied if and only if the types `args-variant-type` and `ops2-variant-type` are well-formed.

3. The exposition-only function template `let-async-scope-bind` is equal to:

```
auto& args = state.args.emplace<decayed - tuple<scope - token - type, Args...>>(
        state.scope.get_token(), std::forward<Args>(args)...);
try {
    auto sndr2 = state.scope.nest(apply(std::move(state.fn), args));
    auto join_sender = state.scope.join();
    auto result_sender = when_all_with_variant(std::move(sndr2), std::move(join_sender)) |
                         then([](auto& result, auto&) { return result; });
    auto rcvr2 = receiver2{std::move(rcvr), std::move(state.env)};
    auto mkop2 = [&] { return connect(std::move(result_sender), std::move(rcvr2)); };
    auto& op2 = state.ops2.emplace<decltype(mkop2())>(emplace-from{mkop2});
    start(op2);
} catch (...) {
    auto result_sender = when_all(just_error(std::current_exception()), state.scope.join());
    auto rcvr2 = receiver2{std::move(rcvr), std::move(state.env)};
    auto mkop2 = [&] { return connect(std::move(result_sender), std::move(rcvr2)); };
    auto& op2 = state.ops2.emplace<decltype(mkop2())>(emplace-from{mkop2});
    start(op2);
}
```

4. `impls-for<decayed-typeof<let_async_scope>>::complete` is is initialized with a callable object equivalent to the following:

```
[]<class Tag, class... Args>
  (auto, auto& state, auto& rcvr, Tag, Args&&... args) noexcept -> void {
    if constexpr (same_as<Tag, decayed-typeof<set_value>>) {
      TRY-EVAL(std::move(rcvr), let-async-scope-bind(state, rcvr, std::forward<Args>(args)...));
    } else {
      Tag()(std::move(rcvr), std::forward<Args>(args)...);
    }
  }
```

5. Let `sndr` and `env` be subexpressions, and let `Sndr` be `decltype((sndr))`. If `sender-for<Sndr, decayed-typeof<let_async_scope>>` is `false`, then the expression `let_async_scope.transform_env(sndr, env)` is ill-formed. Otherwise, it is equal to `JOIN-ENV(let-env(sndr), FWD-ENV(env))`.

6. Let the subexpression `out_sndr` denote the result of the invocation `let_async_scope(sndr, f)` or an object copied or moved from such, and let the subexpression `rcvr` denote a receiver such that the expression `connect(out_sndr, rcvr)` is well-formed. The expression `connect(out_sndr, rcvr)` has undefined behavior unless it creates an asynchronous operation ([async.ops]) that, when started:

    - invokes `f` when `set_value` is called with `sndr`'s result datums,
    - makes its completion dependent on the completion of a sender returned by `f`, and
    - propagates the other completion operations sent by `sndr`.

Acknowledgements

Thanks to Ian Petersen, Lewis Baker, Inbal Levi, Kirk Shoop, Eric Niebler, Ruslan Arutyunyan, Maikel Nadolski, Lucian Radu Teodorescu, and everyone else who contributed to discussions leading to this paper, and commented on early drafts.